FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

Sistema integral de gestión de Biblioteca

Autores:

Garrido Mantilla, Yohan Javier (100%)
Rodríguez Iñigua, Fabricio Leonardo (100%)
Ordinola Rodríguez, Diego Miguel (100%)
Quiñonez Morón, Julio Marcelo (100%)
Jose Fernando Flores Quintana (100%)

Curso:

Técnicas de programación orientada a objetos

Docente del curso:

Mg. Jhon Paul Anampa García

12/11/2026
Lima – Perú
2026-II

1. Resumen del proyecto:

Las bibliotecas pequeñas y medianas que controlan libros, usuarios y préstamos en cuadernos u hojas sueltas enfrentan errores de registro, pérdida de información y falta de trazabilidad sobre quién tiene cada libro y desde cuándo. Para atender este problema se desarrolló el **Sistema integral de gestión de Biblioteca**, una aplicación de escritorio en Java con interfaz gráfica Swing, organizada según el patrón MVC y construida con programación orientada a objetos.

El sistema beneficia al bibliotecario, que centraliza el trabajo y reduce errores; a los lectores, que reciben una atención más rápida; y a la institución, que dispone de información confiable sobre su inventario. Sus principales funcionalidades son la gestión de libros con búsqueda dinámica y control de stock, el registro de usuarios con bloqueo por mora, y los préstamos y devoluciones con fecha automática, plazo de 7 días, baja por pérdida y reposición. Los datos se guardan en archivos CSV y los errores de negocio se manejan con excepciones personalizadas.

2. Descripción del problema:

Situación problemática

En una biblioteca, llevar el control de los libros, usuarios, préstamos y devoluciones de manera manual (cuadernos, hojas sueltas o planillas aisladas) genera errores de registro y pérdida de información. Además, resulta difícil saber qué libros están disponibles, quién tiene un libro prestado y cuándo debe devolverlo.

El problema se presenta en bibliotecas pequeñas y medianas (escolares, municipales o institucionales) que no cuentan con un sistema automatizado. Los antecedentes revisados muestran que este escenario es frecuente tanto en el Perú como en otros países de la región.

usuario involucrado:

- Bibliotecario o administrador: registra libros, usuarios, préstamos y devoluciones, y es quien más sufre la carga de trabajo manual.
- Usuario o lectores: Solicitan libros y esperan que la búsqueda y el préstamo sean rápidos y confiables.
- Institución: necesita información confiable sobre su inventario y el uso de sus libros.

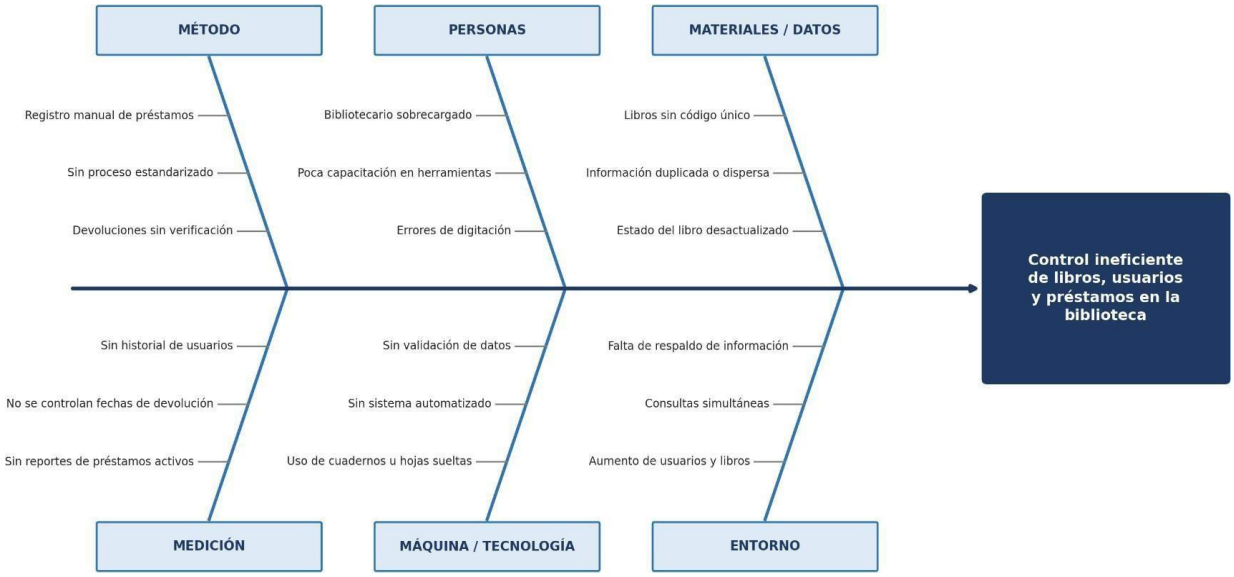
Consecuencias del problema:

- Pérdida de tiempo al buscar libros, usuarios y registros de préstamo.
- Errores y duplicidad de datos, libros cuyo estado real no coincide con el registro.
- Falta de trazabilidad: no se sabe con rapidez quien tiene un libro ni desde cuándo.
- Retrasos o perdidas de libros por falta de control de devolución.
- Menor satisfacción de los usuarios y menor uso de la biblioteca.

Análisis de causa:

Para identificar las causas se utilizó el diagrama de Ishikawa (Ishikawa, 1986) adaptado al contexto de software. El efecto analizado es el control ineficiente de libros, usuarios y préstamos en la biblioteca.

Diagrama de Ishikawa de problema de control de biblioteca:



Principales causas identificadas:

- Método: El registro manual de préstamo y la ausencia de un proceso estandarizado provoca errores y devoluciones sin verificar.
- Materiales/datos: Libros sin código único, información dispersa y estado desactualizado impide sabes que está disponible.
- Medición: No existen reportes de prestamos activos ni control de fechas, por lo que no se detecta retrasos.
- Maquina/tecnología: El uso de cuadernos u hojas sueltas, sin validaciones, es la causa técnica principal que este proyecto busca resolver.
- Personas y entorno: La sobrecarga de bibliotecario y el crecimiento de usuarios y libros agrava el problema.

Antecedentes:

La automatización de control de préstamos bibliotecarios ha sido abordada en varios trabajos que coinciden en que el manejo manual genera pérdida de tiempo e información y que un sistema informático mejora la gestión.

En el ámbito internacional, Matamoros y Sánchez (2011) diagnosticaron la biblioteca municipal del catón Biblián (Ecuador) y encontraron que los prestamos se registraban a mano, lo que obligaba al usuario a esperar mientras se buscaba el material y disminuía su interés por asistir. Su aplicación web para el control de préstamos y consultas permitió a la biblioteca controlar mejor los libros y a los usuarios consultarlos en línea.

En el Perú, Ruiz loro (2021) diseño un sistema web con software libre para la biblioteca de la institución educativa Federico Villarreal (Piura). Su estudio hallo que el 70% de los encuestados no estaba satisfecho con el sistema de vigente y que el 85% consideraba necesario un sistema que optimizara los procesos.

Finalmente, Quispe-Lukaña y Ulloa-Gallardo (2024) implementaron un sistema web para optimizar el préstamo de libros en bibliotecas de Tambopata. Con un diseño preexperimental, reportaron que el tiempo de gestión bajo de 18,4 a 9,6 horas por procesos y que la percepción de eficiencia y la facilidad de uso aumento 45%

Estos trabajos confirman la vigencia del problema. A diferencia de ellos, que platean soluciones web, este proyecto desarrolla un prototipo de consola en java con programación Orientada a Objetos.

Síntesis de los antecedentes revisados:

Autor y año	Contexto	Solución	Resultado principal
Matamoros y Sánchez (2011)	Biblioteca municipal, Biblián (Ecuador)	Aplicación web de préstamos y consultas	Mejor control de libros y consulta en línea
Ruiz Loro (2021)	I. E. Federico Villarreal, Piura (Perú)	Diseño de sistema web con software libre	70 % insatisfecho; 85 % ve necesario un sistema
Quispe-Lukaña y Ulloa-Gallardo (2024)	Bibliotecas de Tambopata (Perú)	Sistema web de préstamos	Tiempo de 18,4 h a 9,6 h; +45 % eficiencia percibida

3. Objetivos y alcance del proyecto:

3.1 objetivo general:

Desarrollar un sistema de información en Java orientado a objetos, integrando una **Interfaz Gráfica de Usuario (GUI)**, que automatice la administración del inventario de libros, el registro de usuarios y el control de préstamos y devoluciones, reduciendo los errores del procesamiento manual y garantizando la trazabilidad de los ejemplares.

3.2 Objetivos específicos:

- OE1 - Estructurar el Modelo POO: diseñar e implementar las clases de dominio fundamentales (Libro, Usuario, Préstamo y el controlador de la biblioteca) aplicando encapsulamiento, abstracción y composición.
- OE2 - Construir la Interfaz Gráfica (GUI): diseñar ventanas gráficas interactivas que reemplacen la consola, facilitando la visualización del catálogo, la búsqueda de libros y la captura de datos.
- OE3 - Gestión Interactiva de Inventario y Usuarios: implementar las operaciones CRUD para libros y usuarios, con búsqueda insensible a mayúsculas y minúsculas.
- OE4 - Automatización del Proceso de Préstamos y Devoluciones: registrar la transacción asociando usuario y libro con fecha automática y actualizando dinámicamente la disponibilidad del libro.
- OE5 - Robustez y Validación de Datos: aplicar excepciones personalizadas y validaciones para que las entradas de los formularios no afecten el flujo de ejecución

3.3 Alcance del sistema:

- Módulo de Gestión de Libros: catálogo en tabla con estado de disponibilidad, búsqueda dinámica, modificación y eliminación; gestión del stock de ejemplares.
- Módulo de Usuarios: registro de lectores, consulta, modificación, eliminación y estado de bloqueo por mora.
- Módulo de préstamos y Devoluciones: registro de préstamos con fecha

automática, devolución con control de plazo (7 días), baja por pérdida y reposición.
- Interfaz Gráfica (Swing): ventana principal con cuatro pestañas, tablas, formularios y ventanas emergentes.
- Persistencia local: exportación automática de archivos CSV al cerrar la aplicación.
- Control de Versiones y Trabajo Colaborativo: repositorio en GitHub con una rama por integrante e integración mediante Pull Requests.

3.4 Restricciones y limitaciones Identificadas:

- Persistencia de datos en memoria volátil (Sin SGBD)
- Curva de aprendizaje e integración de Interfaz Gráfica
- Disponibilidad de información y tiempo de desarrollo reducido
- Validación de tipos de datos en la entrada del usuario
- Conflictos durante la integración de código en la GUI

Impacto en el proyecto:

- Los datos registrados (libros usuarios, préstamos) se pierden al cerrar la aplicación Java
- Interfaces armadas sin orden generan código acoplado.
- El cronograma académico restringe la ventana de tiempo para implementar la GUI y realizar pruebas completas
- Valores inválidos pueden disparar NumberFormatException o NullPointerException.
- El trabajo en paralelo sobre componentes de la interfaz gráfica por diferentes integrantes puede acarrear colisiones o bloqueos al fusionar

Estrategia de Afrontamiento:

- Guardado en archivos locales CSV al cerrar la ventana y carga al iniciar
- Patrón MVC: separar la lógica de negocio (Controlador) de la vista (Swing); reutilizar el modelo ya probado.
- Metodología ágil: migración de consola a GUI dividida por componentes entre los 4 integrantes.
- Bloques try-catch en la vista, validación de campos vacíos y excepciones personalizadas.
- Ramas personales, Pull Requests revisados por al menos 2 colaboradores y no programar sobre main

4. Descripción de la solución:

El **Sistema Integral de Gestión de Biblioteca** es una aplicación de escritorio desarrollada en Java con interfaz gráfica Swing, organizada bajo el patrón MVC y basada en programación orientada a objetos.

Su propósito es facilitar la administración de una biblioteca mediante el control de libros, usuarios, préstamos y devoluciones, evitando los errores y la pérdida de información que pueden producirse con los registros manuales.

Los principales usuarios del sistema son los bibliotecarios, quienes se encargan de registrar y administrar la información, y los lectores, quienes reciben una atención más rápida y organizada. Asimismo, beneficia a la institución al permitirle mantener un mejor control de su inventario.

El sistema contribuye a atender el problema mediante formularios con validaciones, códigos automáticos para identificar libros, usuarios y préstamos, actualización automática del stock y control de fechas de devolución. También permite detectar retrasos, bloquear usuarios con mora y consultar el historial de préstamos. La información se almacena en archivos CSV.

El flujo general de funcionamiento comienza al abrir la aplicación, donde se cargan los libros y usuarios registrados. Luego, el bibliotecario puede registrar libros, gestionar el stock y registrar lectores. Cuando realiza un préstamo, el sistema verifica la disponibilidad del libro y el estado del usuario, registra automáticamente la fecha y descuenta un ejemplar. Al realizar una devolución, calcula los días transcurridos y determina si hubo retraso. Finalmente, al cerrar la aplicación, se exporta la información a archivos CSV.

Prototipo inicial de java:

Clases implementadas:

Clase	Responsabilidad
libro	Representa un libro: código, título, autor y disponibilidad (atributo privado disponible). Métodos prestar(), devolver() y toString(). Desarrollada por Julio
usuario	Representa a un miembro de la biblioteca: código y nombre, con sus métodos de consulta (getters). Desarrollada por Fabrizio y Julio
prestamo	Asocia un libro y un usuario con un código, la fecha automática (LocalDate.now()) y un estado activo/devuelto; método finalizarPrestamo(). Desarrollada por Yohan
biblioteca	Administra tres colecciones ArrayList (libros, usuarios y préstamos) y ejecuta las operaciones: agregar, buscar, listar, prestar y devolver. Desarrollada por Diego
App	Clase principal (main): crea los datos iniciales y muestra el menú de consola de 8 opciones

Aplicación de los conceptos vistos en clase:

Concepto	Aplicación en el prototipo
Clases y objetos	Cuatro clases del modelo (libro, usuario, prestamo, biblioteca) y la clase App. En App se crean objetos: tres libros (El Principito, Don Quijote, Cien años de soledad) y dos usuarios (Julio y Carlos)
Métodos	agregarLibro(), agregarUsuario(), listarLibros(), listarUsuarios(), realizarPrestamo(), devolverLibro() y listarPrestamos() en biblioteca; prestar() y devolver() en libro; finalizarPrestamo() en prestamo
Sobrecarga de métodos	buscarLibro(int codigo) y buscarLibro(String titulo) en la clase biblioteca: el mismo nombre con distinto tipo de parámetro. La búsqueda por título usa equalsIgnoreCase, por lo que no distingue mayúsculas
Manejo de excepciones	En App, el menú está dentro de un bloque try-catch que captura NumberFormatException cuando el usuario escribe un texto en lugar de un número; el programa muestra «Error: debe ingresar un número válido.» y continúa
Colecciones	Tres ArrayList en biblioteca (libros, usuarios y préstamos), recorridas con for-each para buscar y listar
Encapsulamiento	Atributos privados con métodos de acceso (getCodigo(), getTitulo(), isDisponible(), etc.) y toString() sobrescrito en cada clase

5. Requerimientos funcionales:

Módulo de gestión de libros:

RF01 – Registrar libro

Descripción: Registrar un libro mediante un formulario con título y autor. El código se genera automáticamente (LIB-###) y el libro queda inicialmente sin ejemplares (stock 0).
Prioridad: Alta
OE **(Objetivo específico)**: 1, 3

RF02 – Validar código único de libro
Descripción: No permitir dos libros con el mismo código; mostrar error y no guardar (CodigoDuplicadoException)
Prioridad: Alta
OE: 3, 5

RF03 – Listar libros con su estado
Descripción: Mostrar en una tabla todos los libros con código, título, autor, stock y estado de disponibilidad.
Prioridad: Alta
OE: 2, 3

RF04 – Buscar libro por código
Descripción: Buscar un libro ingresando su código y mostrar sus datos y estado (búsqueda dinámica por coincidencia)
Prioridad: Alta
OE: 3

RF05 – Buscar libro por título
Descripción: Buscar libros por título sin distinguir mayúsculas de minúsculas (“don quijote” encuentra “Don Quijote”); también filtra por autor.
Prioridad: Alta
OE: 3

RF06 – Modificar datos de un libro
Descripción: Editar el título y el autor de un libro existente. El código no puede modificarse.
Prioridad: Media
OE: 3

RF07 – Eliminar libro
Descripción: Eliminar un libro del catálogo siempre que no tenga un préstamo activo; si está prestado, rechazar la operación.
Prioridad: Media
OE: 3, 5

Módulo de Usuarios:
RF08 – Registrar usuario

Descripción: Registrar un lector con nombre y correo; el código se genera automáticamente

(US-###).
Prioridad: Alta
OE: 1, 3

RF09 – Validar código único de usuario
Descripción: No permitir dos usuarios con el mismo código; mostrar mensaje de error
Prioridad: Alta
OE: 3, 5

RF10 – Listar usuarios
Descripción: Mostrar en una tabla los usuarios con código, nombre, correo y estado (Activo / Bloqueado).
Prioridad: Alta
OE: 2, 3

RF11 – Buscar usuario por código
Descripción: Consultar un usuario por código o nombre y mostrar sus datos.
Prioridad: Media
OE: 3

RF12 – Modificar datos de un usuario
Descripción: Editar el nombre y el correo de un usuario. El código no puede modificarse.
Prioridad: Media
OE: 3

RF13 – Eliminar usuario

Descripción: Eliminar un usuario siempre que no tenga préstamos activos; en caso contrario rechazar la operación.
Prioridad: Media
OE: 3, 5

Módulo de Préstamos y Devoluciones:

RF14 – Registrar préstamo

Descripción: Registrar un préstamo asociando usuario y libro; el código de préstamo (PRES-###) y la fecha (fecha actual) se asignan automáticamente.
Prioridad: Alta
OE: 4

RF15 – Impedir el préstamo de un libro no disponible

Descripción: No permitir prestar un libro sin ejemplares disponibles (stock 0); mostrar mensaje (LibroNoDisponibleException)
Prioridad: Alta
OE: 4, 5

RF16 – Verificar existencia de libro y usuario en el préstamo
Descripción: Antes de registrar un préstamo verificar que ambos existan; si no, cancelar e informar el motivo.

Prioridad: Alta
OE: 4, 5

RF17 – Actualizar el estado del libro al prestar
Descripción: Al registrar un préstamo, descontar un ejemplar del stock del libro.
Prioridad: Alta
OE: 4

RF18 – Registrar devolución
Descripción: Registrar la devolución de un préstamo activo; el préstamo pasa a "Devuelto a Tiempo" o "Devuelto con Retraso".
Prioridad: Alta
OE: 4

RF19 – Actualizar el estado del libro al devolver
Descripción: Al registrar la devolución, sumar un ejemplar al stock del libro.
Prioridad: Alta
OE: 4

RF20 – Listar préstamos
Descripción: Mostrar todos los préstamos con código, libro, usuario, fecha y estado.
Prioridad: Alta
OE: 2, 4

RF21 – Listar préstamos activos
Descripción: Filtrar y mostrar solo los préstamos que aún no han sido devueltos.
Prioridad: Media
OE: 4

RF22 – Consultar quién tiene un libro prestado
Descripción: Mostrar el usuario que tiene un libro y desde qué fecha (hoy mediante el buscador de préstamos por libro).
Prioridad: Media
OE: 4

RF23 – Consultar historial de préstamos de un usuario

Descripción: Mostrar los préstamos (activos y devueltos) de un usuario (hoy mediante el buscador de préstamos por usuario).
Prioridad: Media
OE: 4

Interfaz gráfica, validación y persistencia:
RF24 – Ventana principal con navegación por módulos

Descripción: Ventana principal con pestañas para Libros, Stock, Usuarios y Préstamos, y salida controlada de la aplicación.
Prioridad: Alta
OE: 2

RF25 – Validación de datos de entrada en formularios
Descripción: Validar campos obligatorios no vacíos y valores numéricos antes de enviar al modelo; mostrar mensaje claro ante datos inválidos.
Prioridad: Alta
OE: 5

RF26 – Manejo de excepciones personalizadas
Descripción: Controlar errores de negocio (código duplicado, libro no disponible, etc.) con excepciones personalizadas, mostrados en ventanas emergentes sin detener la aplicación.
Prioridad: Alta
OE: 5

RF27 – Confirmación de acciones críticas

Descripción: Solicitar confirmación mediante ventana emergente antes de eliminar un libro o usuario y antes de registrar una devolución.
Prioridad: Baja
OE: 2, 5

RF28 – Carga de datos iniciales
Descripción: Al iniciar, cargar datos de ejemplo o los datos guardados previamente.
Prioridad: Media
OE: 1, 3

RF29 – Guardar y cargar datos en archivo local
Descripción: Guardar libros, usuarios y préstamos en archivos CSV y cargarlos al iniciar. Actualmente se guardan los tres y se cargan libros y usuarios; falta cargar préstamos.
Prioridad: Media
OE: 3

RF30 – Gestionar stock de ejemplares
Descripción: Permitir sumar o restar ejemplares al stock de un libro (pestaña Gestión de Stock). El estado muestra “Disponible (n libros)” o “Agotado”.
Prioridad: Alta
OE: 3, 4

RF31 – Controlar plazo de devolución

Descripción: Al devolver, calcular los días transcurridos; si superan 7 días, registrar “Devuelto con Retraso (n días tarde)”.
Prioridad: alta
OE: 4

RF32 – Bloquear usuario por mora
Descripción: Bloquear automáticamente al usuario con devolución tardía e impedirle nuevos préstamos hasta regularizar su situación.
Prioridad: Alta
OE: 4, 5

RF33 – Registrar baja por pérdida y reposición
Descripción: Registrar la pérdida o daño de un libro prestado (bloquea al usuario y el ejemplar

no regresa al stock) y revertirla si el libro es repuesto.
Prioridad: Media
OE: 4

6. Historias de usuario y criterios de aceptación:

Módulo de gestión de libros:

HU01 (RF01): Como bibliotecario, quiero registrar un libro con su título y autor, para mantener el catálogo de la biblioteca actualizado.

HU02 (RF02): Como bibliotecario, quiero que el sistema rechace libros con un código repetido, para evitar duplicidad de datos en el inventario.

HU03 (RF03): Como bibliotecario, quiero ver en una tabla todos los libros con su estado, para saber de inmediato cuáles están disponibles.

HU04 (RF04): Como bibliotecario, quiero buscar un libro por su código, para atender rápidamente a un lector que lo conoce.

HU05 (RF05): Como bibliotecario, quiero buscar un libro por su título sin importar mayúsculas o minúsculas, para encontrarlo, aunque el título se escriba de forma distinta.

HU06 (RF06): Como bibliotecario, quiero modificar el título o el autor de un libro, para corregir errores de registro.

HU07 (RF07): Como bibliotecario, quiero eliminar un libro del catálogo solo si no está prestado, para retirar libros dados de baja sin perder el control de los préstamos vigentes.

Módulo de gestión de usuarios:

HU08 (RF08): Como bibliotecario, quiero registrar a un lector con su nombre y correo, para que pueda solicitar préstamos.

HU09 (RF09): Como bibliotecario, quiero que el sistema impida registrar dos usuarios con el mismo código, para identificar a cada lector de forma única.

HU10 (RF10): Como bibliotecario, quiero ver la lista de usuarios registrados, para confirmar quiénes están habilitados para pedir libros.

HU11 (RF11): Como bibliotecario, quiero buscar un usuario por su código, para verificar sus datos antes de registrar un préstamo.

HU12 (RF12): Como bibliotecario, quiero modificar el nombre de un usuario, para mantener sus datos correctos.

HU13 (RF13): Como bibliotecario, quiero eliminar a un usuario solo si no tiene préstamos activos, para no perder la trazabilidad de los libros que tiene en su poder.

Módulo de préstamos y Devoluciones:

HU14 (RF14): Como bibliotecario, quiero registrar un préstamo asociando un lector con un libro y con la fecha automática, para dejar constancia de quién se lleva cada libro y desde cuándo.

HU15 (RF15): Como bibliotecario, quiero que el sistema no permita prestar un libro sin ejemplares disponibles, para evitar préstamos imposibles de cumplir.

HU16 (RF16): Como bibliotecario, quiero que el sistema verifique que el libro y el usuario existen antes de prestar, para evitar préstamos con datos inválidos.

HU17 (RF17): Como bibliotecario, quiero que el libro pase a "Prestado" automáticamente al

registrar el préstamo, para que el catálogo refleje siempre su estado real.

HU18 (RF18): Como bibliotecario, quiero registrar la devolución de un libro, para cerrar el préstamo y tener el control de las devoluciones.

HU19 (RF19): Como bibliotecario, quiero que el ejemplar vuelva al stock automáticamente al devolverse, para que otros lectores puedan solicitarlo.

HU20 (RF20): Como bibliotecario, quiero ver el listado de todos los préstamos con su fecha y estado, para tener el registro completo de la actividad de la biblioteca.

HU21 (RF21): Como bibliotecario, quiero ver solo los préstamos activos, para identificar qué libros aún no han sido devueltos.

HU22 (RF22): Como bibliotecario, quiero saber quién tiene un libro prestado y desde cuándo, para ubicarlo y gestionar su devolución.

HU23 (RF23): Como lector, quiero que el bibliotecario pueda consultar el historial de mis préstamos, para conocer qué libros he solicitado y cuáles debo devolver

Interfaces gráficas, validación y persistencia:

HU24 (RF24): Como bibliotecario, quiero una ventana principal con acceso claro a los módulos, para trabajar de forma intuitiva sin usar la consola.

HU25 (RF25): Como bibliotecario, quiero que los formularios validen los datos que escribo, para evitar errores por campos vacíos o valores no numéricos.

HU26 (RF26): Como bibliotecario, quiero recibir mensajes claros cuando ocurre un error, para corregirlo sin que el programa se cierre.

HU27 (RF27): Como bibliotecario, quiero una confirmación antes de eliminar o devolver, para evitar acciones accidentales.

HU28 (RF28): Como bibliotecario, quiero que el sistema inicie con datos de ejemplo o con los

datos guardados, para poder comenzar a trabajar de inmediato.

HU29 (RF29): Como administrador de la institución, quiero que el sistema guarde y recupere los datos desde un archivo local, para no perder el inventario ni los préstamos al cerrar la aplicación.

HU30(RF30): Como bibliotecario, quiero registrar cuántos ejemplares tiene cada libro y poder aumentarlos o reducirlos, para llevar un inventario físico exacto.

HU31(RF31): Como bibliotecario, quiero que el sistema detecte las devoluciones fuera del plazo de 7 días, para identificar a los lectores con retraso.

HU32(RF32): Como bibliotecario, quiero que un lector con mora quede bloqueado automáticamente, para evitar nuevos préstamos hasta que regularice su situación.

HU33 (RF33): Como bibliotecario, quiero registrar la pérdida de un libro y poder revertirla si es repuesto, para mantener correcto el stock y el estado del lector.

Criterios de aceptación:

Los criterios se redactan en formato *Dado – Cuando – Entonces* para las funcionalidades implementadas, de modo que cada uno pueda verificarse ejecutando el sistema. Los casos de prueba de la sección 8 corresponden a estos criterios.

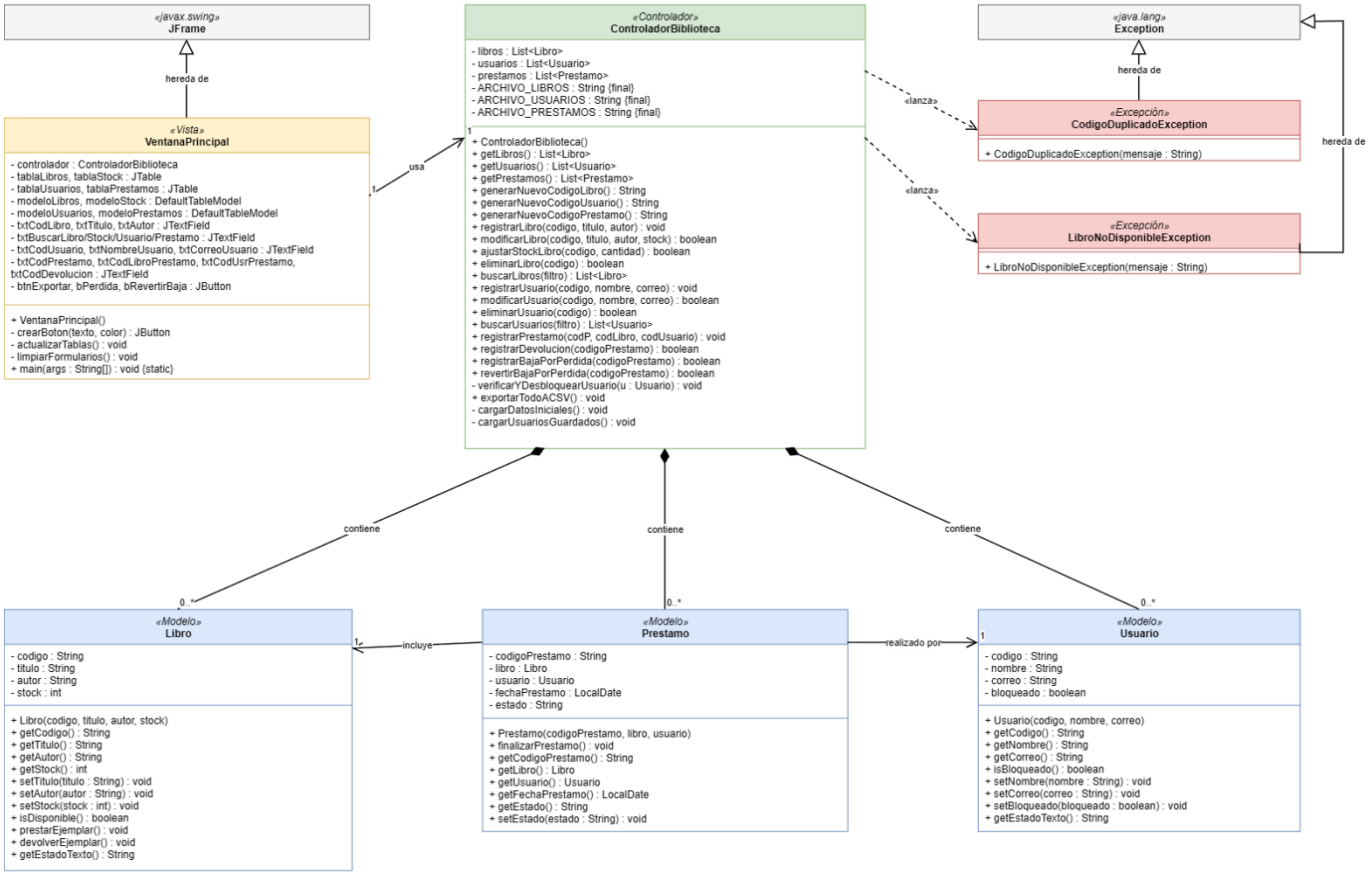
Código	RF	Criterio de aceptación
CA01	RF01, RF02	**Dado** el formulario de libros con un código generado, **cuando** se ingresan título y autor y se pulsa “Registrar Libro”, **entonces** el libro aparece en la tabla con stock 0 y estado “Agotado”, y se muestra “¡Libro registrado exitosamente!”. Si el código ya existiera, se muestra el error y no se guarda.
CA02	RF01, RF08, RF25	**Dado** un formulario con el título, el autor o el nombre vacío, **cuando** se pulsa registrar, **entonces** se muestra un aviso de campos obligatorios y no se registra nada.
CA03	RF03	**Dado** que existen libros registrados, **cuando** se abre la pestaña Catálogo, **entonces** se listan código, título, autor, stock y estado (“Disponible (n libros)” o “Agotado”).
CA04	RF04, RF05	**Dado** el campo “Buscar Libro”, **cuando** se escribe parte de un código, título o autor (en cualquier combinación de mayúsculas), **entonces** la tabla muestra solo los libros que coinciden.
CA05	RF06	**Dado** un libro seleccionado en la tabla, **cuando** se edita su título o autor y se pulsa “Modificar”, **entonces** los cambios se reflejan en la tabla y el código permanece igual.
CA06	RF07	**Dado** un libro seleccionado, **cuando** se pulsa “Eliminar”, **entonces** el libro desaparece de la tabla y se muestra “Eliminado.”. *(La restricción de no eliminar libros con préstamo activo está pendiente.)*
CA07	RF30	**Dado** un libro seleccionado en la pestaña Gestión de Stock, **cuando** se ingresa una cantidad entera (positiva o negativa) y se pulsa “Actualizar / Sumar Stock”, **entonces** el stock se actualiza sin bajar de 0 y se muestra “¡Stock actualizado correctamente!”. Si la cantidad no es un número entero, se muestra un mensaje de error y el programa continúa.
CA08	RF08, RF09, RF10	**Dado** el formulario de usuarios, **cuando** se ingresa nombre y correo y se pulsa “Registrar Usuario”, **entonces** el usuario aparece en la tabla con estado “Activo” y se muestra “¡Usuario registrado!”.

CA09	RF11, RF12	**Dado** un usuario seleccionado o buscado, **cuando** se modifica su nombre o correo, **entonces** la tabla muestra los datos actualizados.
CA10	RF14, RF17	**Dado** un libro con stock mayor que 0 y un usuario activo, **cuando** se ingresan sus códigos y se pulsa “Realizar Préstamo”, **entonces** se crea un préstamo “Activo” con la fecha actual, el stock del libro disminuye en 1 y se muestra “¡Préstamo registrado exitosamente!”.
CA11	RF15	**Dado** un libro con stock 0, **cuando** se intenta prestarlo, **entonces** el sistema muestra que no hay stock disponible y no crea el préstamo.
CA12	RF16	**Dado** un código de libro o de usuario inexistente, **cuando** se pulsa “Realizar Préstamo”, **entonces** se muestra que el libro o el usuario no existen y no se crea el préstamo.
CA13	RF18, RF19, RF31	**Dado** un préstamo activo con 7 días o menos, **cuando** se registra la devolución, **entonces** el estado es “Devuelto a Tiempo” y el stock aumenta en 1.
CA14	RF31, RF32	**Dado** un préstamo activo con más de 7 días, **cuando** se registra la devolución, **entonces** el estado es “Devuelto con Retraso (n días tarde)”, el stock aumenta en 1 y el usuario queda “Bloqueado (Mora)”.
CA15	RF32	**Dado** un usuario bloqueado, **cuando** se intenta registrar un préstamo a su nombre, **entonces** el sistema lo rechaza indicando que el usuario está bloqueado.
CA16	RF33	**Dado** un préstamo activo, **cuando** se pulsa “Registrar Pérdida (Baja)”, **entonces** el estado pasa a “Dado de Baja (Pérdida / Daño)”, el usuario queda bloqueado y el ejemplar no regresa al stock. **Cuando** luego se pulsa “Revertir Baja / Reponer”, **entonces** el ejemplar vuelve al stock y el usuario se desbloquea si no tiene otros retrasos.
CA17	RF20	**Dado** que existen préstamos, **cuando** se abre la pestaña Préstamos, **entonces** se listan código, libro, usuario, fecha y estado, y el buscador filtra por usuario, libro o código.
CA18	RF24, RF26	**Dado** que ocurre un error de negocio o de entrada, **cuando** el sistema lo detecta, **entonces** muestra un mensaje en una ventana emergente y la aplicación sigue funcionando.
CA19	RF28, RF29	**Dado** que la aplicación se inicia por primera vez, **entonces** carga libros y usuarios de ejemplo; **dado** que existen archivos CSV, **entonces** carga los libros y usuarios guardados. **Cuando** se cierra la ventana o se pulsa “Exportar Todo a CSV”, **entonces** se generan los archivos de libros, usuarios y préstamos.

7. Diseño del sistema:

7.1 Diagrama de clases:

Diagrama de clases de la versión actual (3.0), organizado según el patrón MVC: **VentanaPrincipal** (Vista), **ControladorBiblioteca** (Controlador y lógica de negocio) y **Libro**, **Usuario** y **Prestamo** (Modelo), además de las excepciones personalizadas. Se muestran atributos, métodos y relaciones



8. Implementación del sistema:

Funcionalidades desarrolladas:

- Gestión de libros: registro, listado, búsqueda dinámica por código, título o autor, modificación y eliminación.
- Gestión de stock: aumento o disminución de ejemplares con estado “Disponible (n libros)” / “Agotado”.
- Gestión de usuarios: registro, búsqueda, modificación, eliminación y estado de bloqueo.
- Préstamos y devoluciones con fecha automática, control de plazo de 7 días, bloqueo por mora, baja por pérdida y reposición.
- Códigos autogenerados (LIB-###, US-###, PRES-###).
- Persistencia en archivos CSV (exportación al cerrar y al pulsar el botón; carga de libros y usuarios al iniciar).
- Excepciones personalizadas y mensajes emergentes ante errores.

Evolución del proyecto:

En la Entrega 1 se construyó un prototipo de consola con cinco clases (Libro, Usuario, Préstamo, Biblioteca y App) y un menú de 8 opciones, donde cada integrante desarrolló una clase en su rama. En esta entrega el modelo se reorganizó con el patrón MVC y se migró a una interfaz gráfica Swing. Los cambios principales fueron: la clase *Biblioteca* pasó a ser

ControladorBiblioteca; el estado booleano del libro se reemplazó por un stock de libros; se agregaron el correo y el bloqueo del usuario; el préstamo controla plazos y retrasos; y se incorporaron persistencia CSV y excepciones personalizadas

Estructura general del proyecto:

```
Proyecto_Java/
|-- ExcepcionesPersonalizadas/
|     |-- CodigoDuplicadoException.java
|     `-- LibroNoDisponibleException.java
|-- Funcionamiento/
|     |-- Libro.java
|     |-- Usuario.java
|     |-- Prestamo.java
|     `-- ControladorBiblioteca.java
|-- VentanaPrincipal.java   (Vista Swing + main)
`-- README.md

Archivos generados en ejecución:
Inventario_Biblioteca.csv,
Registro_Usuarios.csv, Registro_Prestamos.csv
```

Clases principales:

Clase	Capa	Responsabilidad
Libro	Modelo	Representa un libro (código, título, autor, stock). Métodos prestarEjemplar(), devolverEjemplar(), isDisponible() y getEstadoTexto().
Usuario	Modelo	Representa a un lector (código, nombre, correo, bloqueado). Métodos de acceso y getEstadoTexto().
Prestamo	Modelo	Asocia un Libro y un Usuario con código, fecha (LocalDate.now()) y estado. Al crearse descuenta stock; finalizarPrestamo() calcula el retraso, devuelve el ejemplar y bloquea al usuario si pasaron más de 7 días.
ControladorBiblioteca	Controlador	Administra las colecciones ArrayList y centraliza las reglas de negocio: altas, bajas, modificaciones, búsquedas, préstamos, devoluciones, pérdidas y persistencia CSV.
VentanaPrincipal	Vista	JFrame con cuatro pestañas (Catálogo, Stock, Usuarios, Préstamos), tablas, formularios, buscadores dinámicos y guardado automático al cerrar.
CodigoDuplicadoException	Excepción	Se lanza al registrar un libro o usuario con código ya existente.
LibroNoDisponibleException	Excepción	Se lanza al prestar un libro sin stock o a un usuario bloqueado.

Tecnologías utilizadas:

- **Lenguaje:** Java (programación orientada a objetos).
- **Interfaz gráfica:** Java Swing (JFrame, JTabbedPane, JTable, JOptionPane) con patrón MVC.

- **API estándar:** java.util (ArrayList, List), java.time (LocalDate, ChronoUnit), java.io (lectura y escritura de CSV).
- **Control de versiones y colaboración:** Git y GitHub (ramas, Pull Requests, resolución de conflictos).

Aplicación de conceptos POO:

Concepto	Aplicación en el sistema
Clases y objetos	Clases de dominio Libro, Usuario y Prestamo; los objetos se crean al registrar datos y al cargar los CSV.
Encapsulamiento	Atributos privados con getters y setters; la lógica de stock está dentro de Libro (prestarEjemplar, devolverEjemplar) y no en la vista.
Asociación y composición	Prestamo referencia a un Libro y a un Usuario; ControladorBiblioteca contiene las colecciones del modelo.
Colecciones	Tres ArrayList (libros, usuarios, préstamos) recorridas con for-each y removeIf para buscar, filtrar y eliminar.
Excepciones	Excepciones personalizadas (checked) con throws, try-catch en la vista y NumberFormatException controlada en el ajuste de stock.
Herencia	VentanaPrincipal extends JFrame; las excepciones personalizadas extienden Exception.
Sobrescritura y abstracción	Los métodos getEstadoTexto() ocultan a la vista el detalle del estado; la vista no conoce las reglas del negocio.

Código Fuente – Mejora de proyecto:

```
package ExcepcionesPersonalizadas;

/**
 * Excepción personalizada que se lanza cuando se intenta registrar un
elemento
 * (como un libro o usuario) con un código que ya existe en el sistema.
 *
 * @author Sistema Biblioteca
 * @version 3.0
 */
public class CodigoDuplicadoException extends Exception {

    /**
     * Constructor de la excepción.
     * @param mensaje Descripción detallada del error de duplicidad.
     */
    public CodigoDuplicadoException(String mensaje) {
        super(mensaje);
    }
}
package ExcepcionesPersonalizadas;

/**
```

```java
 * Excepción personalizada que se lanza al intentar realizar un préstamo
 * de un libro que ya se encuentra con estado "Prestado".
 *
 * @author Sistema Biblioteca
 * @version 3.0
 */
public class LibroNoDisponibleException extends Exception {

    /**
     * Constructor de la excepción.
     * @param mensaje Descripción detallada del motivo por el cual no
está disponible.
     */
    public LibroNoDisponibleException(String mensaje) {
        super(mensaje);
    }
}
package Funcionamiento;

/**
 * Entidad que representa a un usuario, incluyendo su estado de bloqueo
por retrasos.
 *
 * @author Sistema Biblioteca
 * @version 3.0
 */
public class Usuario {
    private String codigo;
    private String nombre;
    private String correo;
    private boolean bloqueado; // NUEVO: Estado de bloqueo por mora

    public Usuario(String codigo, String nombre, String correo) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.correo = correo;
        this.bloqueado = false;
    }

    public String getCodigo() { return codigo; }
    public String getNombre() { return nombre; }
    public String getCorreo() { return correo; }
    public boolean isBloqueado() { return bloqueado; }

    public void setNombre(String nombre) { this.nombre = nombre; }
    public void setCorreo(String correo) { this.correo = correo; }
    public void setBloqueado(boolean bloqueado) { this.bloqueado =
bloqueado; }
```

```
    public String getEstadoTexto() {
        return bloqueado ? "Bloqueado (Mora)" : "Activo";
    }
}
package Funcionamiento;

/**
 * Entidad que representa un libro dentro del inventario de la
biblioteca.
 *
 * @author Sistema Biblioteca
 * @version 3.0
 */
public class Libro {
    private String codigo;
    private String titulo;
    private String autor;
    private int stock;

    public Libro(String codigo, String titulo, String autor, int stock) {
        this.codigo = codigo;
        this.titulo = titulo;
        this.autor = autor;
        this.stock = stock;
    }

    public String getCodigo() { return codigo; }
    public String getTitulo() { return titulo; }
    public String getAutor() { return autor; }
    public int getStock() { return stock; }

    public void setTitulo(String titulo) { this.titulo = titulo; }
    public void setAutor(String autor) { this.autor = autor; }
    public void setStock(int stock) { this.stock = stock; }

    public boolean isDisponible() {
        return stock > 0;
    }

    public void prestarEjemplar() {
        if (stock > 0) stock--;
    }

    public void devolverEjemplar() {
        stock++;
    }

    public String getEstadoTexto() {
        if (stock > 0) {
```

```
            String palabra = (stock == 1) ? "libro" : "libros";
            return "Disponible (" + stock + " " + palabra + ")";
        } else {
            return "Agotado";
        }
    }
}
package Funcionamiento;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

/**
 * Entidad transaccional que gestiona plazos, retrasos y bloqueo de
usuarios.
 *
 * @author Sistema Biblioteca
 * @version 3.0
 */
public class Prestamo {
    private String codigoPrestamo;
    private Libro libro;
    private Usuario usuario;
    private LocalDate fechaPrestamo;
    private String estado;

    public Prestamo(String codigoPrestamo, Libro libro, Usuario usuario)
{
        this.codigoPrestamo = codigoPrestamo;
        this.libro = libro;
        this.usuario = usuario;
        this.fechaPrestamo = LocalDate.now();
        this.estado = "Activo";
        this.libro.prestarEjemplar();
    }

    /**
     * Constructor para restaurar un préstamo guardado en CSV.
     * No descuenta stock: el inventario guardado ya refleja ese
préstamo.
     */
    public Prestamo(String codigoPrestamo, Libro libro, Usuario usuario,
LocalDate fechaPrestamo, String estado) {
        this.codigoPrestamo = codigoPrestamo;
        this.libro = libro;
        this.usuario = usuario;
        this.fechaPrestamo = fechaPrestamo;
        this.estado = estado;
    }
```

```
    public void finalizarPrestamo() {
        LocalDate fechaDevolucion = LocalDate.now();
        long diasTranscurridos = ChronoUnit.DAYS.between(fechaPrestamo,
fechaDevolucion);

        if (diasTranscurridos > 7) {
            long diasTarde = diasTranscurridos - 7;
            this.estado = "Devuelto con Retraso (" + diasTarde + " días
tarde)";
            // El usuario queda bloqueado automáticamente si se pasó de
los 7 días
            this.usuario.setBloqueado(true);
        } else {
            this.estado = "Devuelto a Tiempo";
        }
        this.libro.devolverEjemplar();
    }

    public String getCodigoPrestamo() { return codigoPrestamo; }
    public Libro getLibro() { return libro; }
    public Usuario getUsuario() { return usuario; }
    public LocalDate getFechaPrestamo() { return fechaPrestamo; }
    public String getEstado() { return estado; }
    public void setEstado(String estado) { this.estado = estado; }
}
package Funcionamiento;

import ExcepcionesPersonalizadas.CodigoDuplicadoException;
import ExcepcionesPersonalizadas.LibroNoDisponibleException;

import java.io.*;
import java.nio.ByteBuffer;
import java.nio.charset.CharacterCodingException;
import java.nio.charset.Charset;
import java.nio.charset.CodingErrorAction;
import java.nio.charset.StandardCharsets;
import java.nio.file.AtomicMoveNotSupportedException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import java.util.ArrayList;
import java.util.List;
import java.time.LocalDate;
import java.time.format.DateTimeParseException;
import java.time.temporal.ChronoUnit;

/**
```

```java
 * Controlador principal que centraliza la lógica de negocio, reglas de
préstamos,
 * control de stock, bloqueo por mora y persistencia automática en
archivos CSV.
 *
 * @author Sistema Biblioteca
 * @version 3.0
 */
public class ControladorBiblioteca {
    private List<Libro> libros = new ArrayList<>();
    private List<Usuario> usuarios = new ArrayList<>();
    private List<Prestamo> prestamos = new ArrayList<>();

    private final String ARCHIVO_LIBROS = "Inventario_Biblioteca.csv";
    private final String ARCHIVO_USUARIOS = "Registro_Usuarios.csv";
    private final String ARCHIVO_PRESTAMOS = "Registro_Prestamos.csv";

    // Si algún CSV existe pero no se pudo leer, no se exporta para no
sobrescribirlo con datos vacíos.
    private boolean errorDeCarga = false;

    public ControladorBiblioteca() {
        cargarDatosIniciales();
        cargarUsuariosGuardados();
        cargarPrestamosGuardados();

        if (usuarios.isEmpty()) {
            usuarios.add(new Usuario("US-001", "Maria Gutierrez",
"maria@mail.com"));
            usuarios.add(new Usuario("US-002", "Valentino Morales",
"valentino@mail.com"));
        }
    }

    public List<Libro> getLibros() { return libros; }
    public List<Usuario> getUsuarios() { return usuarios; }
    public List<Prestamo> getPrestamos() { return prestamos; }

    public String generarNuevoCodigoLibro() {
        int mayor = 0;
        for (Libro l : libros) {
            try {
                String[] partes = l.getCodigo().split("-");
                int num = Integer.parseInt(partes[1]);
                if (num > mayor) mayor = num;
            } catch (Exception e) {}
        }
        return String.format("LIB-%03d", mayor + 1);
    }
```

```
    public String generarNuevoCodigoUsuario() {
        int mayor = 0;
        for (Usuario u : usuarios) {
            try {
                String[] partes = u.getCodigo().split("-");
                int num = Integer.parseInt(partes[1]);
                if (num > mayor) mayor = num;
            } catch (Exception e) {}
        }
        return String.format("US-%03d", mayor + 1);
    }

    public String generarNuevoCodigoPrestamo() {
        int mayor = 0;
        for (Prestamo p : prestamos) {
            try {
                int num = Integer.parseInt(p.getCodigoPrestamo().split("-
")[1]);
                if (num > mayor) mayor = num;
            } catch (Exception e) {}
        }
        return String.format("PRES-%03d", mayor + 1);
    }

    /**
     * Registra un nuevo libro en el catálogo con stock inicial en 0.
     */
    public void registrarLibro(String codigo, String titulo, String
autor) throws CodigoDuplicadoException {
        for (Libro l : libros) {
            if (l.getCodigo().equalsIgnoreCase(codigo)) {
                throw new CodigoDuplicadoException("El código " + codigo
+ " ya existe.");
            }
        }
        libros.add(new Libro(codigo, titulo, autor, 0));
    }

    public boolean modificarLibro(String codigo, String nuevoTitulo,
String nuevoAutor, int stockActual) {
        for (Libro l : libros) {
            if (l.getCodigo().equalsIgnoreCase(codigo)) {
                l.setTitulo(nuevoTitulo);
                l.setAutor(nuevoAutor);
                l.setStock(stockActual);
                return true;
            }
        }
```

```
        return false;
    }

    public boolean ajustarStockLibro(String codigo, int cantidadAñadir) {
        for (Libro l : libros) {
            if (l.getCodigo().equalsIgnoreCase(codigo)) {
                int nuevoStock = l.getStock() + cantidadAñadir;
                if (nuevoStock < 0) nuevoStock = 0;
                l.setStock(nuevoStock);
                return true;
            }
        }
        return false;
    }

    public boolean eliminarLibro(String codigo) {
        return libros.removeIf(l ->
l.getCodigo().equalsIgnoreCase(codigo));
    }

    public List<Libro> buscarLibros(String filtro) {
        List<Libro> resultados = new ArrayList<>();
        String query = filtro.toLowerCase().trim();
        for (Libro l : libros) {
            if (l.getCodigo().toLowerCase().contains(query) ||
                l.getTitulo().toLowerCase().contains(query) ||
                l.getAutor().toLowerCase().contains(query)) {
                resultados.add(l);
            }
        }
        return resultados;
    }

    public void registrarUsuario(String codigo, String nombre, String
correo) throws CodigoDuplicadoException {
        for (Usuario u : usuarios) {
            if (u.getCodigo().equalsIgnoreCase(codigo)) {
                throw new CodigoDuplicadoException("El código de usuario
" + codigo + " ya existe.");
            }
        }
        usuarios.add(new Usuario(codigo, nombre, correo));
    }

    public boolean modificarUsuario(String codigo, String nuevoNombre,
String nuevoCorreo) {
        for (Usuario u : usuarios) {
            if (u.getCodigo().equalsIgnoreCase(codigo)) {
                u.setNombre(nuevoNombre);
```

```
                u.setCorreo(nuevoCorreo);
                return true;
            }
        }
        return false;
    }

    public boolean eliminarUsuario(String codigo) {
        return usuarios.removeIf(u ->
u.getCodigo().equalsIgnoreCase(codigo));
    }

    public List<Usuario> buscarUsuarios(String filtro) {
        List<Usuario> resultados = new ArrayList<>();
        String query = filtro.toLowerCase().trim();
        for (Usuario u : usuarios) {
            if (u.getCodigo().toLowerCase().contains(query) ||
                u.getNombre().toLowerCase().contains(query)) {
                resultados.add(u);
            }
        }
        return resultados;
    }

    /**
     * Valida disponibilidad y que el usuario no esté bloqueado antes de
realizar el préstamo.
     */
    public void registrarPrestamo(String codP, String codLibro, String
codUsuario) throws LibroNoDisponibleException, Exception {
        Libro libroEncontrado = null;
        Usuario usuarioEncontrado = null;

        for (Libro l : libros) { if
(l.getCodigo().equalsIgnoreCase(codLibro)) { libroEncontrado = l; break;
} }
        for (Usuario u : usuarios) { if
(u.getCodigo().equalsIgnoreCase(codUsuario)) { usuarioEncontrado = u;
break; } }

        if (libroEncontrado == null || usuarioEncontrado == null) {
            throw new Exception("El Libro o el Usuario especificado no
existen.");
        }
        if (usuarioEncontrado.isBloqueado()) {
            throw new LibroNoDisponibleException("El usuario '" +
usuarioEncontrado.getNombre() + "' está BLOQUEADO por multas o
retrasos.");
        }
```

```
        if (!libroEncontrado.isDisponible()) {
            throw new LibroNoDisponibleException("No hay stock disponible
del libro '" + libroEncontrado.getTitulo() + "'.");
        }

        prestamos.add(new Prestamo(codP, libroEncontrado,
usuarioEncontrado));
    }

    public boolean registrarDevolucion(String codigoPrestamo) {
        for (Prestamo p : prestamos) {
            if (p.getCodigoPrestamo().equalsIgnoreCase(codigoPrestamo) &&
p.getEstado().startsWith("Activo")) {
                LocalDate hoy = LocalDate.now();
                long diasPasados =
ChronoUnit.DAYS.between(p.getFechaPrestamo(), hoy);

                p.finalizarPrestamo();

                if (diasPasados > 7) {
                    p.getUsuario().setBloqueado(true);
                } else {
                    verificarYDesbloquearUsuario(p.getUsuario());
                }
                return true;
            }
        }
        return false;
    }

    public boolean registrarBajaPorPerdida(String codigoPrestamo) {
        for (Prestamo p : prestamos) {
            if (p.getCodigoPrestamo().equalsIgnoreCase(codigoPrestamo) &&
p.getEstado().startsWith("Activo")) {
                p.setEstado("Dado de Baja (Pérdida / Daño)");
                p.getUsuario().setBloqueado(true);
                return true;
            }
        }
        return false;
    }

    public boolean revertirBajaPorPerdida(String codigoPrestamo) {
        for (Prestamo p : prestamos) {
            if (p.getCodigoPrestamo().equalsIgnoreCase(codigoPrestamo) &&
p.getEstado().startsWith("Dado de Baja")) {
                p.setEstado("Devuelto tras Baja (Repuesto)");
                p.getLibro().devolverEjemplar();
                verificarYDesbloquearUsuario(p.getUsuario());
```

```
                return true;
            }
        }
        return false;
    }

    private void verificarYDesbloquearUsuario(Usuario u) {
        boolean tieneRetrasosPendientes = false;
        for (Prestamo p : prestamos) {
            if
(p.getUsuario().getCodigo().equalsIgnoreCase(u.getCodigo()) &&
p.getEstado().contains("Retraso")) {
                tieneRetrasosPendientes = true;
                break;
            }
        }
        if (!tieneRetrasosPendientes) {
            u.setBloqueado(false);
        }
    }

    //
==========================================================================
    //  PERSISTENCIA CSV
    //  - UTF-8 con BOM (Excel muestra bien tildes y ñ)
    //  - Campos con ';' o '"' se entrecomillan, así no se corrompen las
columnas
    //  - Escritura en archivo temporal + reemplazo, para no dejar
archivos a medias
    //
==========================================================================
    private static final char SEP = ';';
    private static final String BOM = "\uFEFF";
    private static final String FIN_LINEA = "\r\n";

    private static String esc(String s) {
        if (s == null) return "";
        s = s.replace("\r", " ").replace("\n", " ");
        if (s.indexOf(SEP) >= 0 || s.indexOf('"') >= 0) {
            s = "\"" + s.replace("\"", "\"\"") + "\"";
        }
        return s;
    }

    private static String fila(String... campos) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < campos.length; i++) {
            if (i > 0) sb.append(SEP);
            sb.append(esc(campos[i]));
```

```
        }
        return sb.toString();
    }

    private static List<String> separar(String linea) {
        List<String> campos = new ArrayList<>();
        StringBuilder actual = new StringBuilder();
        boolean enComillas = false;
        for (int i = 0; i < linea.length(); i++) {
            char c = linea.charAt(i);
            if (enComillas) {
                if (c == '"') {
                    if (i + 1 < linea.length() && linea.charAt(i + 1) ==
'"') { actual.append('"'); i++; }
                    else enComillas = false;
                } else {
                    actual.append(c);
                }
            } else if (c == '"') {
                enComillas = true;
            } else if (c == SEP) {
                campos.add(actual.toString());
                actual.setLength(0);
            } else {
                actual.append(c);
            }
        }
        campos.add(actual.toString());
        return campos;
    }

    private void escribirCSV(String nombre, String cabecera, List<String>
filas) throws IOException {
        Path destino = Paths.get(nombre).toAbsolutePath();
        Path temporal = destino.resolveSibling(nombre + ".tmp");
        try (BufferedWriter w = Files.newBufferedWriter(temporal,
StandardCharsets.UTF_8)) {
            w.write(BOM);
            w.write(cabecera);
            w.write(FIN_LINEA);
            for (String f : filas) {
                w.write(f);
                w.write(FIN_LINEA);
            }
        }
        try {
            Files.move(temporal, destino,
StandardCopyOption.REPLACE_EXISTING, StandardCopyOption.ATOMIC_MOVE);
        } catch (AtomicMoveNotSupportedException e) {
```

```
            Files.move(temporal, destino,
StandardCopyOption.REPLACE_EXISTING);
        }
    }

    /** Lee un CSV (sin la cabecera). Acepta UTF-8 con/sin BOM y, si no
es UTF-8 válido, Windows-1252. */
    private List<List<String>> leerCSV(String nombre) throws IOException
{
        List<List<String>> filas = new ArrayList<>();
        File archivo = new File(nombre);
        if (!archivo.exists()) return filas;

        byte[] bytes = Files.readAllBytes(archivo.toPath());
        String texto;
        try {
            texto = StandardCharsets.UTF_8.newDecoder()
                    .onMalformedInput(CodingErrorAction.REPORT)
                    .onUnmappableCharacter(CodingErrorAction.REPORT)
                    .decode(ByteBuffer.wrap(bytes)).toString();
        } catch (CharacterCodingException e) {
            texto = new String(bytes, Charset.forName("windows-1252"));
        }
        if (texto.startsWith(BOM)) texto = texto.substring(1);

        boolean cabecera = true;
        for (String linea : texto.split("\\r?\\n")) {
            if (cabecera) { cabecera = false; continue; }
            if (linea.trim().isEmpty()) continue;
            filas.add(separar(linea));
        }
        return filas;
    }

    /**
     * Guarda libros, usuarios y préstamos en sus archivos CSV.
     * @throws IOException si algún archivo no se pudo escribir (p. ej.
está abierto en Excel).
     */
    public void exportarTodoACSV() throws IOException {
        if (errorDeCarga) {
            throw new IOException("No se exportó para no sobrescribir
archivos CSV que no pudieron leerse al iniciar. "
                    + "Revisa Inventario_Biblioteca.csv,
Registro_Usuarios.csv y Registro_Prestamos.csv.");
        }

        List<String> filas = new ArrayList<>();
        for (Libro l : libros) {
```

```
            filas.add(fila(l.getCodigo(), l.getTitulo(), l.getAutor(),
String.valueOf(l.getStock()), l.getEstadoTexto()));
        }
        escribirCSV(ARCHIVO_LIBROS, "Código;Título;Autor;Stock;Estado",
filas);

        filas = new ArrayList<>();
        for (Usuario u : usuarios) {
            filas.add(fila(u.getCodigo(), u.getNombre(), u.getCorreo(),
u.getEstadoTexto()));
        }
        escribirCSV(ARCHIVO_USUARIOS,
"Código;Nombre;Correo;EstadoBloqueo", filas);

        filas = new ArrayList<>();
        for (Prestamo p : prestamos) {
            filas.add(fila(p.getCodigoPrestamo(),
                    p.getLibro().getCodigo(), p.getLibro().getTitulo(),
                    p.getUsuario().getCodigo(),
p.getUsuario().getNombre(),
                    p.getFechaPrestamo().toString(), p.getEstado()));
        }
        escribirCSV(ARCHIVO_PRESTAMOS,
"CodigoPrestamo;CodigoLibro;Libro;CodigoUsuario;Usuario;FechaPrestamo;Est
ado", filas);
    }

    /** Ruta absoluta de la carpeta donde se guardan los CSV (para
mostrarla al usuario). */
    public String getRutaExportacion() {
        return new File(ARCHIVO_LIBROS).getAbsoluteFile().getParent();
    }

    private void cargarDatosIniciales() {
        if (!new File(ARCHIVO_LIBROS).exists()) {
            libros.add(new Libro("LIB-001", "El Señor de los Anillos",
"John Ronald Reuel Tolkien", 3));
            libros.add(new Libro("LIB-002", "Harry Potter", "Joanne
Rowling", 2));
            return;
        }
        try {
            for (List<String> p : leerCSV(ARCHIVO_LIBROS)) {
                if (p.size() >= 4) {
                    try {
                        libros.add(new Libro(p.get(0), p.get(1),
p.get(2), Integer.parseInt(p.get(3).trim())));
                    } catch (NumberFormatException e) {
```

```
                        System.err.println("Fila de inventario ignorada
(stock inválido): " + p);
                    }
                }
            }
        } catch (IOException e) {
            errorDeCarga = true;
            System.err.println("No se pudo leer " + ARCHIVO_LIBROS + ": "
+ e.getMessage());
        }
    }

    private void cargarUsuariosGuardados() {
        try {
            for (List<String> p : leerCSV(ARCHIVO_USUARIOS)) {
                if (p.size() >= 3) {
                    Usuario u = new Usuario(p.get(0), p.get(1),
p.get(2));
                    if (p.size() >= 4 && p.get(3).contains("Bloqueado"))
{
                        u.setBloqueado(true);
                    }
                    usuarios.add(u);
                }
            }
        } catch (IOException e) {
            errorDeCarga = true;
            System.err.println("No se pudo leer " + ARCHIVO_USUARIOS +
": " + e.getMessage());
        }
    }

    /**
     * Restaura los préstamos guardados. Antes se exportaban pero nunca
se volvían a cargar,
     * por lo que tras reiniciar el stock seguía descontado y no se podía
registrar la devolución.
     * Acepta el formato nuevo (7 columnas) y el antiguo (5 columnas, por
título y nombre).
     */
    private void cargarPrestamosGuardados() {
        try {
            for (List<String> p : leerCSV(ARCHIVO_PRESTAMOS)) {
                Libro libro; Usuario usuario; String fecha, estado;
                if (p.size() >= 7) {
                    libro = buscarLibroPorCodigo(p.get(1));
                    usuario = buscarUsuarioPorCodigo(p.get(3));
                    fecha = p.get(5); estado = p.get(6);
                } else if (p.size() >= 5) {
```

```
                    libro = buscarLibroPorTitulo(p.get(1));
                    usuario = buscarUsuarioPorNombre(p.get(2));
                    fecha = p.get(3); estado = p.get(4);
                } else {
                    continue;
                }
                if (libro == null || usuario == null) {
                    System.err.println("Préstamo ignorado (libro o
usuario no existe): " + p);
                    continue;
                }
                try {
                    prestamos.add(new Prestamo(p.get(0), libro, usuario,
LocalDate.parse(fecha.trim()), estado));
                } catch (DateTimeParseException e) {
                    System.err.println("Préstamo ignorado (fecha
inválida): " + p);
                }
            }
        } catch (IOException e) {
            errorDeCarga = true;
            System.err.println("No se pudo leer " + ARCHIVO_PRESTAMOS +
": " + e.getMessage());
        }
    }

    private Libro buscarLibroPorCodigo(String codigo) {
        for (Libro l : libros) if
(l.getCodigo().equalsIgnoreCase(codigo.trim())) return l;
        return null;
    }

    private Libro buscarLibroPorTitulo(String titulo) {
        for (Libro l : libros) if
(l.getTitulo().equalsIgnoreCase(titulo.trim())) return l;
        return null;
    }

    private Usuario buscarUsuarioPorCodigo(String codigo) {
        for (Usuario u : usuarios) if
(u.getCodigo().equalsIgnoreCase(codigo.trim())) return u;
        return null;
    }

    private Usuario buscarUsuarioPorNombre(String nombre) {
        for (Usuario u : usuarios) if
(u.getNombre().equalsIgnoreCase(nombre.trim())) return u;
        return null;
    }
```

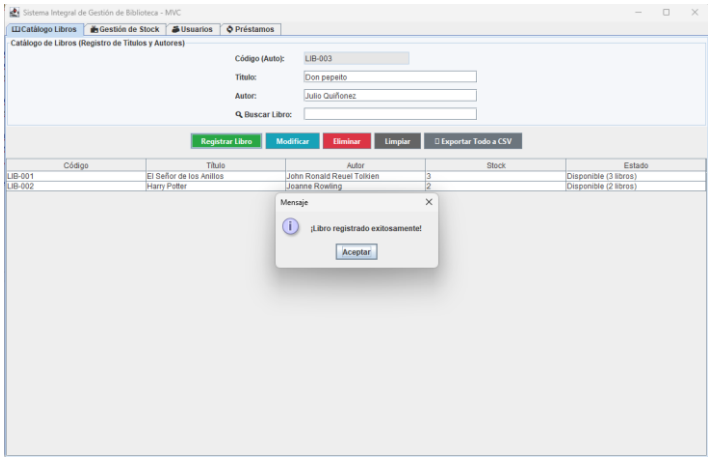
}

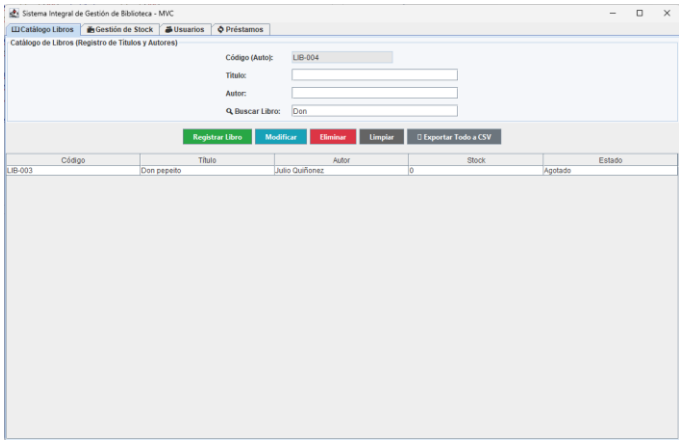
9. Pruebas y resultados:

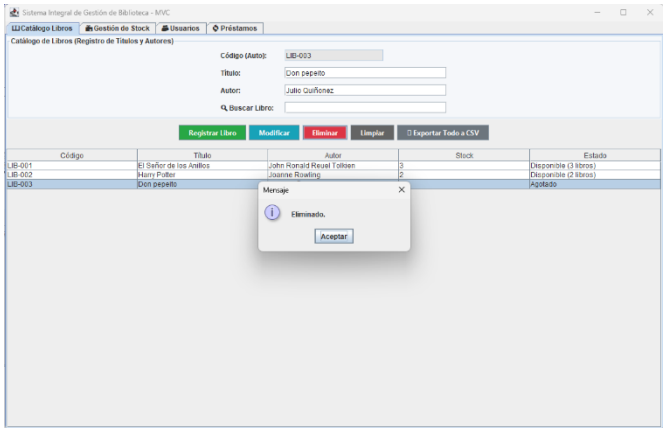
ID	Caso de prueba	Entrada	Resultado esperado	Resultado
CP01	Registrar libro	Título y autor válidos	Libro en tabla y mensaje de éxito	Aprobado
CP02	Buscar libro por texto	“Don”	Solo coincidencias en la tabla	Aprobado
CP03	Modificar libro	Nuevo título/autor	Datos actualizados	Aprobado
CP04	Eliminar libro	Libro seleccionado	Libro retirado y mensaje	Aprobado
CP05	Sumar stock	+4 al libro con 3	Stock 7, “Disponible (7 libros)”	Aprobado
CP06	Registrar usuario	Nombre y correo	Usuario en tabla y mensaje	Aprobado
CP07	Eliminar usuario	Usuario seleccionado	Usuario retirado	Aprobado
CP08	Registrar préstamo	LIB-001 y US-003	Préstamo PRES-001 creado	Aprobado
CP09	Registrar pérdida	Préstamo activo	Baja registrada; usuario bloqueado	Aprobado
CP10	Revertir baja	Préstamo dado de baja	Ejemplar repuesto; usuario desbloqueado	Aprobado

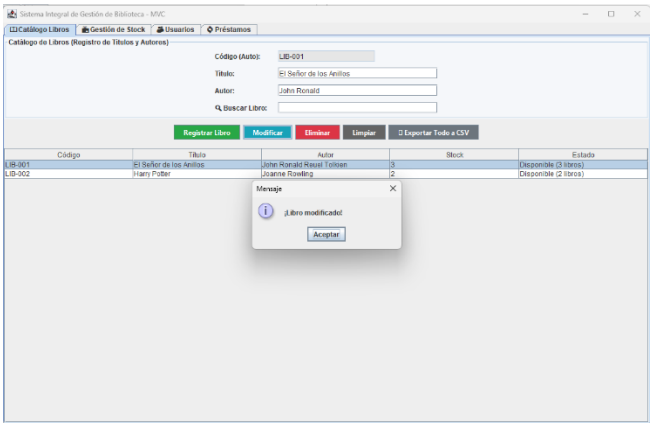
10.Evidencia del funcionamiento del sistema:

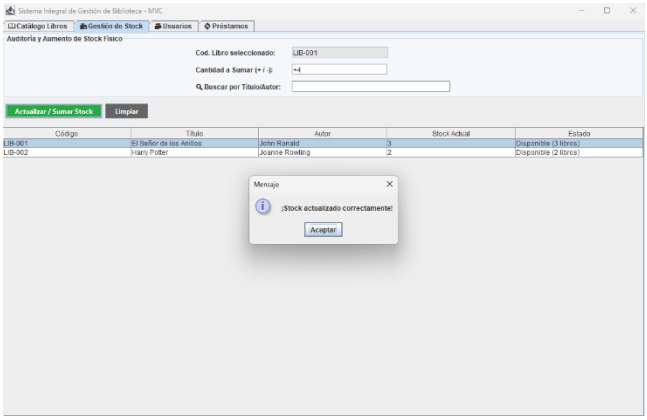
Capturas de la aplicación Swing ejecutándose con las operaciones principales de cada módulo

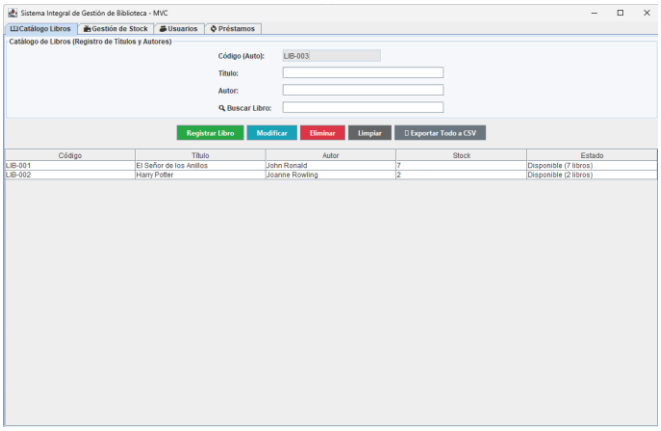


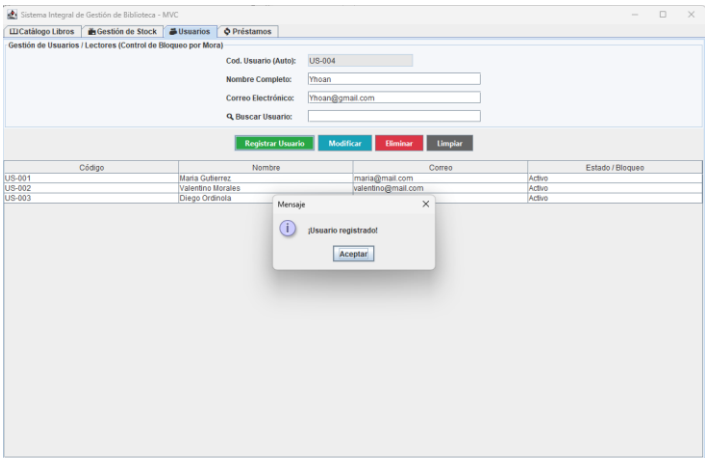


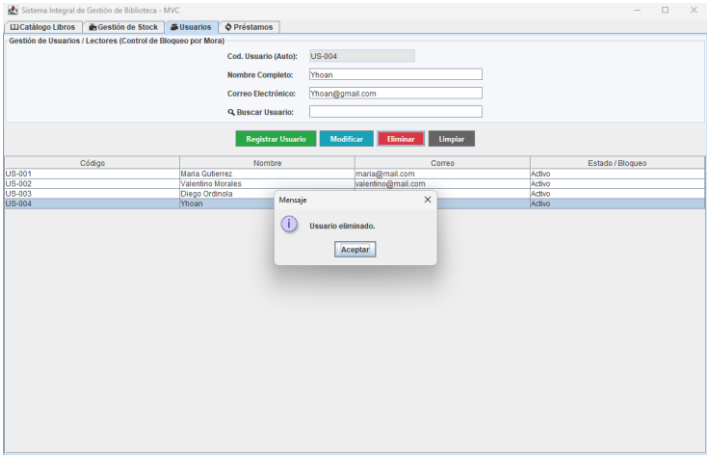


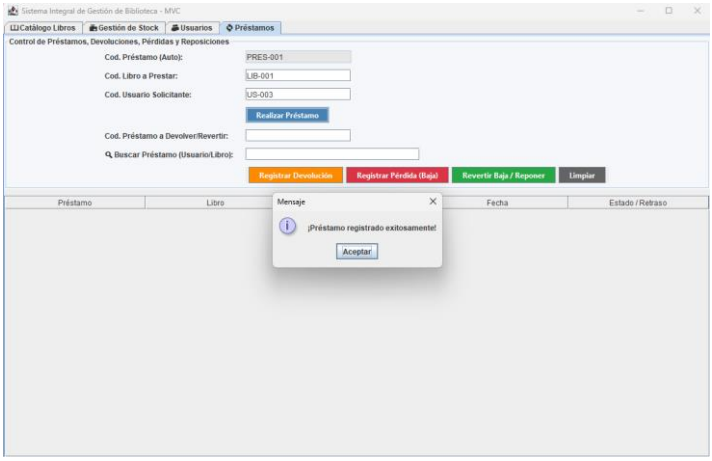


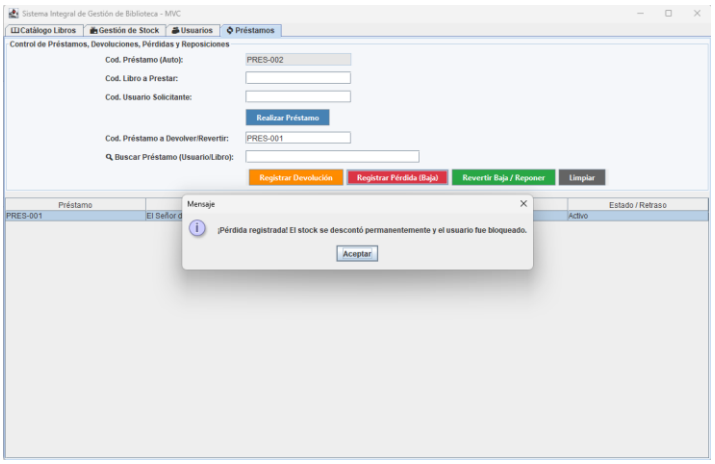

Sistema Integral de Gestión de Biblioteca - MVC
Catálogo Libros
Gestión de Stock
Usuarios
Préstamos
Auditoría y Aumento de Stock Físico
Cod. Libro seleccionado:
LIB-001
Cantidad a Sumar (+ / -):
+4
Buscar por Título/Autor:
Actualizar / Sumar Stock
Limpiar
Código
Título
Autor
Stock Actual
Estado
LIB-001
El Señor de los Anillos
John Ronald
3
Disponible (3 libros)
LIB-002
Harry Potter
Joanne Rowling
2
Disponible (2 libros)
Mensaje
¡Stock actualizado correctamente!
Aceptar

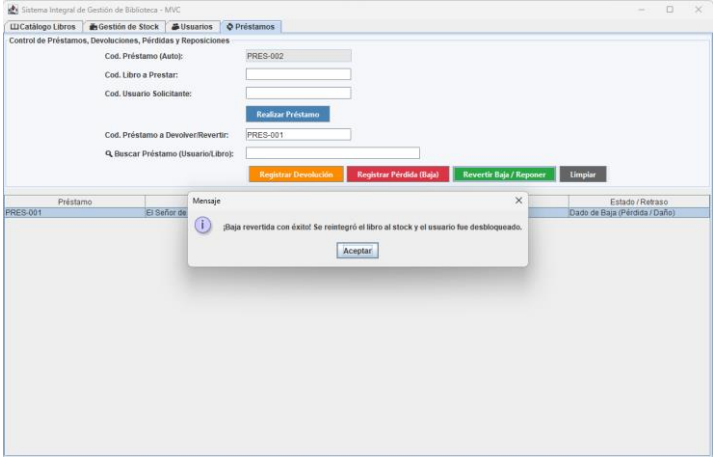

Sistema Integral de Gestión de Biblioteca - MVC
Catálogo Libros
Gestión de Stock
Usuarios
Préstamos
Catálogo de Libros (Registro de Títulos y Autores)
Código (Auto):
LIB-003
Título:
Autor:
Buscar Libro:
Registrar Libro
Modificar
Eliminar
Limpiar
Exportar Todo a CSV
Código
Título
Autor
Stock
Estado
LIB-001
El Señor de los Anillos
John Ronald
7
Disponible (7 libros)
LIB-002
Harry Potter
Joanne Rowling
2
Disponible (2 libros)


Sistema Integral de Gestión de Biblioteca - MVC
Catálogo Libros
Gestión de Stock
Usuarios
Préstamos
Gestión de Usuarios / Lectores (Control de Bloqueo por Mora)
Cod. Usuario (Auto):
US-004
Nombre Completo:
Yhoan
Correo Electrónico:
Yhoan@gmail.com
Buscar Usuario:
Registrar Usuario
Modificar
Eliminar
Limpiar
Código
Nombre
Correo
Estado / Bloqueo
US-001
Maria Gutierrez
maria@mail.com
Activo
US-002
Valentino Morales
valentino@mail.com
Activo
US-003
Diego Ordinola
Activo
Mensaje
¡Usuario registrado!
Aceptar


Sistema Integral de Gestión de Biblioteca - MVC
Catálogo Libros
Gestión de Stock
Usuarios
Préstamos
Gestión de Usuarios / Lectores (Control de Bloqueo por Mora)
Cod. Usuario (Auto):
US-004
Nombre Completo:
Yhoan
Correo Electrónico:
Yhoan@gmail.com
Buscar Usuario:
Registrar Usuario
Modificar
Eliminar
Limpiar
Código
Nombre
Correo
Estado / Bloqueo
US-001
Maria Gutierrez
maria@mail.com
Activo
US-002
Valentino Morales
valentino@mail.com
Activo
US-003
Diego Ordinola
Activo
US-004
Yhoan
Activo
Mensaje
Usuario eliminado.
Aceptar


Sistema Integral de Gestión de Biblioteca - MVC
Catálogo Libros
Gestión de Stock
Usuarios
Préstamos
Control de Préstamos, Devoluciones, Pérdidas y Reposiciones
Cod. Préstamo (Auto):
PRES-001
Cod. Libro a Prestar:
LIB-001
Cod. Usuario Solicitante:
US-003
Realizar Préstamo
Cod. Préstamo a Devolver/Revertir:
Buscar Préstamo (Usuario/Libro):
Registrar Devolución
Registrar Pérdida (Baja)
Revertir Baja / Reponer
Limpiar
Préstamo
Libro
Fecha
Estado / Retraso
Mensaje
¡Préstamo registrado exitosamente!
Aceptar


Sistema Integral de Gestión de Biblioteca - MVC
Catálogo Libros
Gestión de Stock
Usuarios
Préstamos
Control de Préstamos, Devoluciones, Pérdidas y Reposiciones
Cod. Préstamo (Auto):
PRES-002
Cod. Libro a Prestar:
Cod. Usuario Solicitante:
Realizar Préstamo
Cod. Préstamo a Devolver/Revertir:
PRES-001
Buscar Préstamo (Usuario/Libro):
Registrar Devolución
Registrar Pérdida (Baja)
Revertir Baja / Reponer
Limpiar
Préstamo
Estado / Retraso
PRES-001
Activo
Mensaje
¡Pérdida registrada! El stock se descontó permanentemente y el usuario fue bloqueado.
Aceptar


Sistema Integral de Gestión de Biblioteca - MVC
Catálogo Libros
Gestión de Stock
Usuarios
Préstamos
Control de Préstamos, Devoluciones, Pérdidas y Reposiciones
Cod. Préstamo (Auto):
PRES-002
Cod. Libro a Prestar:
Cod. Usuario Solicitante:
Realizar Préstamo
Cod. Préstamo a Devolver/Revertir:
PRES-001
Buscar Préstamo (Usuario/Libro):
Registrar Devolución
Registrar Pérdida (Baja)
Revertir Baja / Reponer
Limpiar
Préstamo
Estado / Retraso
PRES-001
Dado de Baja (Pérdida / Daño)
Mensaje
¡Baja revertida con éxito! Se reintegró el libro al stock y el usuario fue desbloqueado.
Aceptar

11.Trabajo colaborativo:

URL del repositorio:

https://github.com/Juli0-Q/Proyecto_Java.git

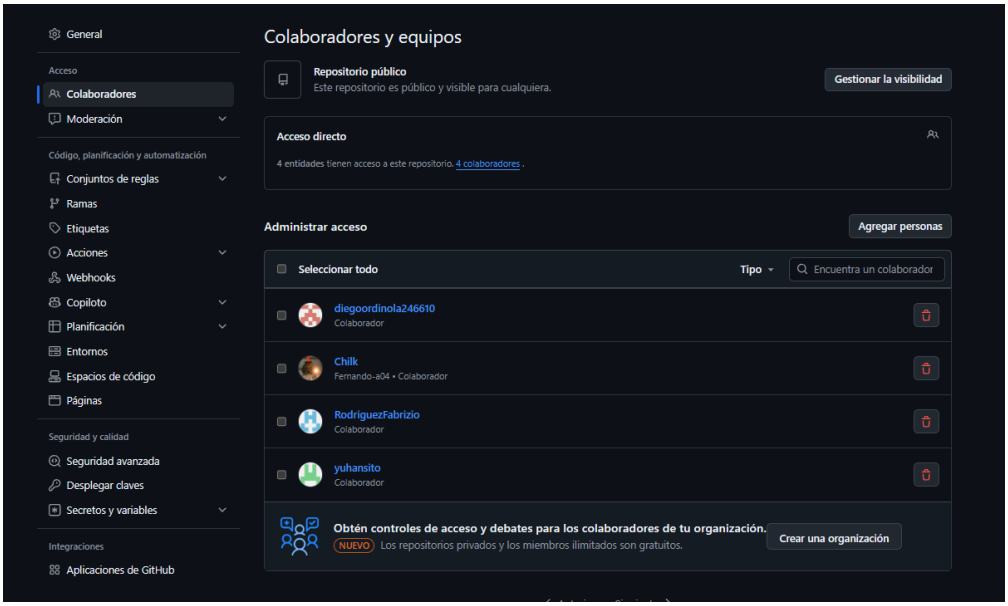
organización del trabajo:

El repositorio se creó el 13 de septiembre de 2026 y el trabajo se organizó con una rama personal por integrante, más la rama main, que contiene la versión que funciona y donde nadie programa directamente.

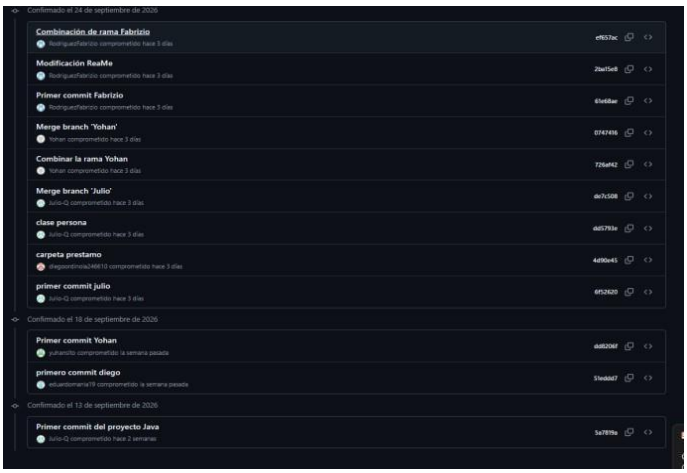
Integrantes, usuarios de GitHub y clases asignadas:

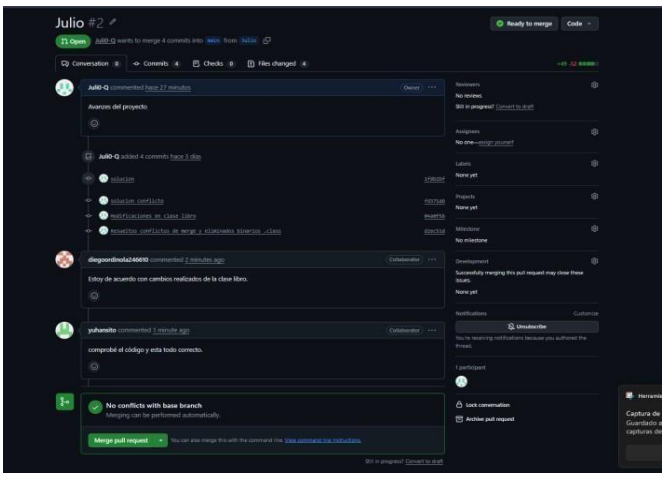
Integrante	Usuario de GitHub	Rama	Clase a cargo
Garrido Mantilla, Yohan Javier	yuhansito	Yohan	prestamo
Ordinola Rodriguez, Diego Miguel	diegoordinola246610	Diego	biblioteca
Quiñonez Morón, Julio Marcelo	Juli0-Q (propietario)	Julio	libro
Rodriguez Iñigua, Fabrizio Leonardo	RodriguezFabrizio	Fabrizio	usuario
Jose Fernando Flores Quintana	Fernando-a04	Fernando	Excepciones

Colaboradores del repositorio:



Evidencia de historial de commits y pull request:





Practica de control de versión aplicadas:

- Ramas individuales y fusión (merge): cada integrante trabajó en su rama y la integró a main.
- Resolución de conflictos: se resolvieron conflictos en README.md y en Libro.java revisando línea por línea.
- Revisión de código con pull requests: el pull request n.º 2 fue revisado por dos colaboradores antes de la fusión.
- Repositorio limpio: se eliminaron del control de versiones los archivos compilados (.class) y se recomienda excluir /bin/ con. gitignore.
- Git stash: el equipo investigó su uso para guardar cambios sin confirmar al cambiar de rama.

12.Conclusiones:

Se desarrolló un Sistema Integral de Gestión de Biblioteca en Java con interfaz gráfica Swing, que permite administrar libros, usuarios, stock, préstamos y devoluciones de manera más organizada.

Se cumplieron las principales funcionalidades propuestas, como el registro de información, la búsqueda, la modificación, la eliminación y el control de préstamos, facilitando la administración de la biblioteca.

Durante el desarrollo se aplicaron conceptos de programación orientada a objetos, como clases, objetos, encapsulamiento, abstracción y composición, permitiendo organizar mejor el código y las responsabilidades del sistema.

El trabajo permitió fortalecer los conocimientos de Java, el diseño de interfaces gráficas, la validación de datos y el uso de GitHub para desarrollar un proyecto en equipo.

El sistema todavía presenta limitaciones en la persistencia de información y algunas funcionalidades pendientes, por lo que puede seguir mejorándose en futuras versiones.

13. Recomendaciones y mejoras futuras:

- Implementar una base de datos para almacenar y recuperar toda la información, incluyendo los préstamos, sin depender únicamente de archivos CSV.
- Completar el filtro de préstamos activos para facilitar el seguimiento de los libros que todavía no han sido devueltos.
- Mejorar las validaciones de los formularios para controlar datos incorrectos, campos vacíos y registros duplicados.
- Incorporar reportes de préstamos, devoluciones, retrasos y disponibilidad de libros.
- Implementar diferentes niveles de acceso para bibliotecarios y administradores.

14. Referencias:

Ishikawa, K. (1986). Guide to quality control (2.ª ed. rev.). Asian Productivity Organization.

Matamoros Motoche, M. A., & Sánchez Idrovo, M. F. (2011). Análisis, diseño y creación de una aplicación web para el control de préstamos y consultas en la Biblioteca de la Municipalidad del cantón Biblián [Tesis de grado, Universidad Politécnica Salesiana]. Repositorio Institucional UPS. https://dspace.ups.edu.ec/handle/123456789/2103

Quispe-Lukaña, D. A., & Ulloa-Gallardo, N. J. (2024). Desarrollo e implementación de un sistema web para optimizar los procesos de préstamo de libros de una biblioteca, Tambopata. Revista Amazonía Digital, 3(2), e288. https://revistas.unamad.edu.pe/index.php/rad/article/view/288

Ruiz Loro, C. E. (2021). Diseño de un sistema web con software libre para la gestión de biblioteca en la institución educativa Federico Villarreal - Piura; 2018 [Trabajo de investigación, Universidad Católica Los Ángeles de Chimbote]. Repositorio Institucional ULADECH. https://hdl.handle.net/20.500.13032/22813

Oracle. (s. f.-a). *Creating a GUI with Swing*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/

Oracle. (s. f.-b). *How to use tables*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/components/table.html

Oracle. (s. f.-c). *Lesson: Exceptions*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/essential/exceptions/

Oracle. (s. f.-d). *Java Platform, Standard Edition 21 API specification*. https://docs.oracle.com/en/java/javase/21/docs/api/

Oracle. (s. f.-e). *Packaging programs in JAR files*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/deployment/jar/

Shafranovich, Y. (2005). *Common format and MIME type for comma-separated values (CSV) files* (RFC 4180). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc4180

Visual Studio Code. (s. f.). *Java in Visual Studio Code*.
https://code.visualstudio.com/docs/languages/java

15. Anexos

Instrucciones para la ejecución:

Al abrirlo ves una ventana con **4 pestañas** arriba. Cada una tiene un formulario arriba y una tabla abajo. Los campos grises son automáticos, no se tocan. Si haces clic en una fila de la tabla, sus datos se cargan solos en el formulario.

Catálogo Libros

Aquí agregas los títulos.

1. Escribe el **Título** y el **Autor**. El código (LIB-001, LIB-002…) se genera solo.
2. Dale a **Registrar Libro**. Ojo: el libro nace con **stock 0**, el stock se sube en la otra pestaña.
3. Para **Modificar** o **Eliminar**, haz clic en el libro en la tabla, cambia lo que quieras y pulsa el botón.
4. En **Buscar Libro** escribes y la tabla se filtra al instante por código, título o autor.
5. **Limpiar** vacía el formulario y deja la tabla completa otra vez.
6. **Exportar Todo a CSV** guarda todo en los archivos CSV al momento. También se guarda solo al cerrar la ventana con la X.

Gestión de Stock

Aquí le pones cuántos ejemplares hay de cada libro.

1. Haz clic en el libro en la tabla (se llena “Cod. Libro seleccionado”).
2. En **Cantidad a Sumar (+ / -)** pon un número entero. Positivo suma (ej. 5) y negativo resta (ej. -2).
3. Pulsa **Actualizar / Sumar Stock**. El stock nunca baja de 0.
4. Tiene su propio buscador por título o autor.

Usuarios

Aquí registras a los lectores.

1. Escribe **Nombre Completo** (obligatorio) y **Correo**. El código US-001, US-002… es automático.
2. **Registrar Usuario** lo agrega. Para **Modificar** o **Eliminar**, selecciónalo en la tabla primero.
3. Verás si cada usuario está **Activo** o **Bloqueado (Mora)**. Un usuario bloqueado no puede pedir libros.
4. Hay buscador por nombre o código.

Préstamos

Es la pestaña principal del día a día.

Prestar un libro

1. En **Cod. Libro a Prestar** escribe el código (ej. LIB-001) y en **Cod. Usuario Solicitante** el del lector (ej. US-001).
2. Pulsa **Realizar Préstamo**. El código del préstamo (PRES-001…) es automático.
3. Te avisa si no hay stock o si el usuario está bloqueado, y en ese caso no presta.
4. El plazo es de **7 días** y el stock baja en 1.

Devolver un libro

1. Haz clic en el préstamo en la tabla (se llena **Cod. Préstamo a Devolver/Revertir**) o escribe el código.
2. Pulsa **Registrar Devolución**. El stock vuelve a subir en 1.
3. Si pasaron más de 7 días, el préstamo queda "Devuelto con Retraso" y el usuario **se bloquea** automáticamente.

Si el libro se perdió o dañó

- Selecciona el préstamo y pulsa **Registrar Pérdida (Baja)**. El libro no vuelve al stock y el usuario queda bloqueado.
- Si después lo pagan o lo recuperan, selecciona ese mismo préstamo y pulsa **Revertir Baja / Reponer**. El libro vuelve al stock y se desbloquea al usuario.

Buscar: en **Buscar Préstamo** escribes un usuario o libro y se filtra la tabla.

Tips rápidos

- Orden recomendado la primera vez: registra libros → súmales stock → registra usuarios → ya puedes prestar.
- Antes de **Exportar**, cierra los CSV si los tienes abiertos en Excel, porque si no, no se podrá guardar.
- Cierra siempre con la **X** de la ventana para que se guarde todo.